

Домашнее задание 5. Пайплайны.

Формат сдачи:

Все домашние задания по курсу сдаются как страничка в Github-репозитории, который структурирован так, как указано в конце задания ниже. Пожалуйста, обратите внимание на взаимное расположение файлов в репозитории и на их название. Я буду запускать автоматический базовый чекер, который проверяет наличие файлов в репозитории для каждой домашки. Для каждого задания старайтесь хранить листинг команд, которыми вы пользовались – они понадобятся для сдачи. Часто ответы на задание нужно будет писать в README.md в какой-то папке репозитория (воспринимайте это как что-то похожее на домашки по практической биоинформатике).

Уточнение:

Если у вас не Linux, вы можете воспользоваться либо WSL2 (устанавливается автоматически в Windows > 10), либо установить cygwin: cygwin.org, позволяющий работать в командной строке. По всем вопросам пишите в чат курса или в личные сообщения преподавателям. При работе с git не забывайте как можно чаще коммитить изменения. В markdown можно вставлять картинки и таблицы – не стесняйтесь это делать, если чувствуете, что это хорошо иллюстрирует ситуацию.

Задание 1. Пайплайн для сборки генома.

Реализуйте DAG, который строит полный пайплайн обработки бактериального генома. Пайплайн должен скачивать paired-end FASTQ риды из публичного источника (например SRA или GEO) по идентификатору, затем выполнять de novo сборку генома с использованием SPAdes, запущенного в контейнере. После сборки необходимо построить индекс полученного генома и выполнить выравнивание исходных ридов на собранный референс с помощью картировщика (например, BWA-MEM), после чего получить отсортированный BAM файл (и опционально его индекс). Все шаги должны быть оформлены как отдельные задачи DAG, биоинформатические инструменты должны запускаться в контейнерах, а промежуточные результаты сохраняться либо на диске, либо передаваться через XCom. В отчёт в папку репозитория **hw5/airflow-assembler** положите все файлы, требуемые для воспроизведения (кроме fastq / bam).

Задание 2. Эксперимент MLflow.

Реализуйте ML-пайплайн с использованием MLflow на датасете TidyTuesday (<https://github.com/rfordatascience/tidytuesday/blob/main/data/2026/2026-03-03/readme.md>). Скачайте данные и объедините обе таблицы, после чего постройте модель, предсказывающую количество яиц на основе доступных признаков для имеющихся черепах. Проведите эксперимент в MLflow, включая логирование параметров, метрик и самой модели, а также сравнение как минимум нескольких подходов (например, линейная модель и градиентный бустинг). В отчёт в папку репозитория **hw5/turtles** положите файлы, требуемые для воспроизведения ваших экспериментов (скрипт на питоне, либо ноутбук с воспроизводимым пайплайном обучения), а также сохранённый MLflow run, где видны параметры, метрики и артефакты модели. Можно добавить скриншот экспериментов в MLflow.

Задание 3. Streamlit-интерфейс.

Для ещё одного датасета из публичного репозитория TidyTuesday (<https://github.com/rfordata science/tidyuesday/blob/main/data/2026/2026-03-24/readme.md>) реализуйте веб-приложение на Streamlit, которое работает с первым миллионом десятичных знаков числа π . В интерфейсе должно быть поле ввода, куда пользователь вводит произвольное число, после чего приложение вычисляет, сколько раз это число встречается в заданном отрезке числа π , и выводит результат. Дополнительно приложение должно строить и выводить графики распределения количества вхождений, оценку плотности встречаемости и определять наиболее частую подстроку длины k , в которой встречается заданное число. В отчёт в папку репозитория **hw5/turtles** положите скрипт-приложение, которое можно запустить с помощью **streamlit run app.py**.